

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 6

Inside the xv6 File System

Course: Operating Systems (Track B: xv6)

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **2**
1.1 Laboratory Purpose and Scope . . . 2
1.2 The xv6 Storage Stack Architecture . . . 2

2 Part 1: Read What xv6 Shows You **4**
2.1 Execution Traces and Initial Directory Exploration . . . 4
2.2 Answers and Technical Analysis (Part 1) . . . 4

3 Part 2: Write `mystat` and Hard Link Observation **6**
3.1 Implementation of `user/mystat.c` . . . 6
3.2 Execution Traces: Watching Hard Links at Work . . . 6
3.3 Answers and Technical Analysis (Part 2) . . . 7

4 Part 3: Measure the Limits **9**
4.1 Inspecting Filesystem Definitions and Disk Layout . . . 9
4.2 Answers and Technical Analysis (Part 3) . . . 9

5 Part 4: Write `find` **11**
5.1 Implementation of `user/find.c` . . . 11
5.2 Design Rationale and Safety Measures . . . 12
5.3 Execution Traces and Verification . . . 12
5.4 Stretch Question Analysis . . . 12

6 Conclusions **13**

1 Theoretical Background and Objectives

1.1 Laboratory Purpose and Scope

The xv6 filesystem implementation encapsulates the core architecture of classical UNIX storage subsystems (such as the early UNIX V6 filesystem and modern ext4 principles) within approximately one thousand readable lines of C code. The primary objective of Laboratory Work Nr. 6 is to explore the internal data structures, disk layout, inode representations, and system call implementations of the xv6 filesystem.

Throughout this laboratory, we investigate:

- **Directory Tables**: How directories in UNIX/xv6 are stored as ordinary files containing sequences of fixed-size `struct dirent` entries.
- **Inode Representation and Types**: Decoding the disk inode structure (`struct dinode`), in-memory inode management (`struct inode`), and file types (`T_DIR`, `T_FILE`, `T_DEVICE`).
- **Metadata Inspection Utility (`mystat`)**: Developing a user-space utility to query and display file type, inode number, link count (`nlink`), and byte size using the POSIX-style `stat()` system call.
- **Hard Links and Atomic Unlinking**: Observing hard link reference counts, examining the decrement operation inside `sys_unlink()`, and understanding deferred storage block reclamation via `iput()` and `itrunc()`.
- **Filesystem Layout and Theoretical Limits**: Calculating disk layout blocks (boot, superblock, logging journal, inode table, free bitmap, data blocks) from `nmeta`, understanding filename truncation governed by `DIRSIZ`, and computing maximum addressable file sizes via direct and indirect block pointers.
- **Recursive Hierarchy Walking (`find`)**: Implementing the recursive directory search utility `user/find.c`, handling directory cycles (`.` and `..`), and respecting per-process file descriptor limits (`NOFILE = 16`).

1.2 The xv6 Storage Stack Architecture

The storage subsystem in xv6 is organized in seven clean modular layers:

1. **Disk Layer**: VirtIO block device driver (`kernel/virtio_disk.c`) reading and writing raw 1024-byte disk sectors via DMA.
2. **Buffer Cache Layer**: Synchronized in-memory cache of disk blocks (`kernel/bio.c`) providing caching, mutual exclusion via sleeplocks, and write-back buffering.
3. **Logging Layer**: Write-ahead journaling transaction system (`kernel/log.c`) ensuring atomicity and consistency across multi-step filesystem operations.
4. **Inode Layer**: Index node abstraction (`kernel/fs.c`) managing anonymous file identity, block extent lookup, and reference counting.
5. **Directory Layer**: Directory formatting (`kernel/fs.c`) treating directories as lists of directory entries mapping 14-byte names to 16-bit inode numbers.

6. **Pathname Layer**: Hierarchical path resolution (`namei()`, `nameiparent()`) resolving paths like `/a/b/c` iteratively.

7. **File Descriptor Layer**: POSIX-compatible abstraction table (`kernel/file.c`) managing user file descriptors pointing to open files, pipes, or devices.

2 Part 1: Read What xv6 Shows You

2.1 Execution Traces and Initial Directory Exploration

We launched xv6 within QEMU and analyzed the output produced by the default `ls` command on the root directory and a newly created child directory:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
args           2 3 36576
cat            2 4 38112
echo           2 5 36888
sleep          2 6 36576
forktest       2 7 18248
grep           2 8 41648
init           2 9 37272
kill           2 10 36824
ln             2 11 36616
ls             2 12 40240
mkdir          2 13 36880
rm             2 14 36864
sh             2 15 60824
stressfs       2 16 37728
usertests      2 17 209536
grind          2 18 53360
wc             2 19 39136
zombie         2 20 36144
logstress      2 21 38896
forphan        2 22 37648
dorphan        2 23 37088
sync           2 24 36240
console        3 25 0
$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
```

Each row output by `user/ls.c` formats metadata fields as:

`[name]  [type]  [inode]  [size in bytes]`

2.2 Answers and Technical Analysis (Part 1)

1. **What do types 1, 2 and 3 mean (`kernel/stat.h`)?**
   Answer:
   Header file `kernel/stat.h` defines the integer constants representing inode types in xv6:

```c
#define T_DIR     1   // Directory
#define T_FILE    2   // File
#define T_DEVICE  3   // Device

```

   - **Type 1 (`T_DIR`)**: Directory. A file whose data blocks contain a table of `struct dirent` entries mapping names to inode numbers.

- **Type 2 (`T_FILE`)**: Regular file. Contains uninterpreted byte streams (e.g., text files, binary executables).
- **Type 3 (`T_DEVICE`)**: Character device node (such as `console`). Operations on device nodes bypass filesystem storage blocks and invoke registered kernel driver callbacks.

2. **Which inode number does the root directory have, and why are `.` and `..` the same there?**
   Answer:

   - **Root Inode Number**: The root directory has **inode number 1** (`ROOTINO = 1` defined in `kernel/fs.h`). This is verified by the `ls` output, where both `.` and `..` display inode `1`.
   - **Why `.` and `..` are identical**: `.` refers to the directory itself, while `..` refers to the parent directory. Because the root directory `/` is the absolute apex of the filesystem hierarchy, it has no parent. In standard UNIX filesystem design, the parent reference of the root directory is defined to point back to the root directory itself.

3. **An empty directory `a` has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does `a` contain?**
   Answer:
   In `kernel/fs.h`, an xv6 directory entry is defined as:

```
struct dirent {
  ushort inum;
  char name[DIRSIZ];
};

```

   - `ushort inum`: 2 bytes (16-bit unsigned integer holding the inode number).
   - `char name[DIRSIZ]`: 14 bytes (`DIRSIZ = 14`, fixed-width filename string).
   - **Size of one entry**: 2 bytes + 14 bytes = **16** bytes.
   - **Entries contained in `a`**: When `mkdir a` creates a directory, the kernel initializes it with exactly two entries:
     (a) `.`: pointing to directory `a`'s own inode (inode 26).
     (b) `..`: pointing to parent root directory (inode 1).
     Consequently, the total byte size of an empty directory is:

$$\text{Size} = 2 \times 16 \text{ bytes} = \mathbf{32} \text{ bytes}.$$

3 Part 2: Write `mystat` and Hard Link Observation

3.1 Implementation of `user/mystat.c`

We implemented the user utility `user/mystat.c` to inspect file status via the `stat()` system call:

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t){
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

The program was registered in `Makefile` under `UPROGS`:

```
UPROGS=\
    $U/_args\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    $U/_ls\
    $U/_mystat\
    ...
```

3.2 Execution Traces: Watching Hard Links at Work

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes

$ ln README r2
$ mystat README r2
```

```
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes

$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes

$ cat r2 | wc
48 336 2441
```

3.3 Answers and Technical Analysis (Part 2)

1. **Paste the outputs after `ln` and after `rm`. What changed and what did not?**
   Answer:

   - Output after `ln README r2`:

     ```
     $ mystat README r2
     README: file, inode 2, links 2, size 2441 bytes
     r2: file, inode 2, links 2, size 2441 bytes
     ```

   - Output after `rm README`:

     ```
     $ mystat r2
     r2: file, inode 2, links 1, size 2441 bytes
     ```

   - **What changed**:
     - After `ln`: The link count field (`links`) of inode 2 incremented from 1 to 2. A new directory entry named `r2` was created in the directory pointing to inode 2.
     - After `rm`: The directory entry `README` was unlinked, and the link count (`links`) of inode 2 decremented from 2 to 1.
   - **What did not change**:
     - Inode number remained **2**.
     - File size remained **2441 bytes**.
     - File type remained **file**.
     - The underlying file contents remained identical and fully intact, verified by `cat r2 | wc` outputting 48 lines, 336 words, and 2441 bytes.

2. **Find the line in `kernel/sysfile.c` (`sys_unlink`) that decrements the link count. When are the data blocks freed?**
   Answer:
   In `kernel/sysfile.c`, inside the implementation of `sys_unlink()` (around line 241):

   ```
   ip->nlink--;
   iupdate(ip);
   iunlockput(ip);

   ```

Line 241: `ip->nlink-;` decrements the on-disk link count.
When data blocks are freed: The data blocks are **not** necessarily freed when `sys_unlink()` executes. They are deallocated only when:

(a) The inode's link count reaches zero (`ip->nlink == 0`), **AND**

(b) All processes holding open file descriptors referencing the inode close their descriptors (`ip->ref == 1` when dropping the reference in `iput()`).

When `iput()` in `kernel/fs.c` determines that `ip->valid && ip->nlink == 0 && ip->ref == 1`, it invokes `itrunc(ip)`. The `itrunc()` function iterates over all 12 direct blocks and the 256 indirect blocks, freeing each block back to the allocation bitmap via `bfree()`, zeroes out the file size, and marks the inode type as free (`ip->type = 0`).

3. **xv6 has no symbolic links and no permissions. Look at `struct dinode` in `kernel/fs.h`: which fields does an xv6 inode have?**
Answer:
In `kernel/fs.h`, the on-disk inode data structure is defined as:

```
struct dinode {
  short type;               // File type
  short major;              // Major device number (T_DEVICE only)
  short minor;              // Minor device number (T_DEVICE only)
  short nlink;              // Number of links to inode in file system
  uint size;                // Size of file (bytes)
  uint addrs[NDIRECT + 1]; // Data block addresses
};

```

The fields present in an xv6 inode are:

(a) `type` (`short`, 2 bytes): File type (`T_DIR`, `T_FILE`, `T_DEVICE`, or 0 for unused).

(b) `major` (`short`, 2 bytes): Major device number (identifies device driver).

(c) `minor` (`short`, 2 bytes): Minor device number.

(d) `nlink` (`short`, 2 bytes): Hard link counter tracking directory references.

(e) `size` (`uint`, 4 bytes): Logical file length in bytes.

(f) `addrs[NDIRECT + 1]` (`uint[13]`, 52 bytes): Disk block addresses of file extents (12 direct block numbers + 1 indirect block number).

Total size of an on-disk inode: $2 + 2 + 2 + 2 + 4 + 52 = \mathbf{64}$ bytes.
Architectural comparison: Unlike Linux ext4 inodes, xv6 dinodes contain no owner UID, no group GID, no permission mode bits, no access/modification/change timestamps (`atime`, `mtime`, `ctime`), and no symlink target path strings.

4 Part 3: Measure the Limits

4.1 Inspecting Filesystem Definitions and Disk Layout

We queried filesystem constants in `kernel/fs.h` and examined the disk structure created by `mkfs`:

```
xv6-riscv > grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ"
    kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14

xv6-riscv > make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
    total 2000

$ mkdir averyveryverylongname
$ ls
...
averyveryveryl 1 27 32
```

4.2 Answers and Technical Analysis (Part 3)

1. **What does `ls` show for `averyveryverylongname`, and which constant explains it?**
   Answer:
   - `ls` displays the directory name truncated to **`averyveryveryl`** (14 characters).
   - The constant that explains this is **`#define DIRSIZ 14`** in `kernel/fs.h`.
   - In xv6, each directory entry defines its name buffer as `char name[DIRSIZ]`. When `sys_mkdir()` creates the entry, at most 14 characters are copied into the record. Any excess characters are silently ignored.

2. **Compute the largest file xv6 can hold: 12 direct blocks plus one indirect block that holds block numbers of 4 bytes each, blocks of `BSIZE` bytes. Show the calculation.**
   Answer:
   From `kernel/fs.h`:
   - Block size: BSIZE = 1024 bytes.
   - Address size: sizeof(uint) = 4 bytes.
   - Direct blocks: NDIRECT = 12 blocks.
   - Indirect block capacity:
   $$\text{NINDIRECT} = \frac{\text{BSIZE}}{\text{sizeof(uint)}} = \frac{1024}{4} = 256 \text{ blocks.}$$
   - Total data blocks per file:
   $$\text{Total Blocks} = \text{NDIRECT} + \text{NINDIRECT} = 12 + 256 = \mathbf{268} \text{ blocks.}$$

- Maximum addressable file size:

$$\text{Max File Size} = 268 \times 1024 \text{ bytes} = \mathbf{274,432} \text{ bytes} = \mathbf{268} \text{ KiB}.$$

3. **From the `nmeta` line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?**
Answer:
From the output:

```
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks
    1953 total 2000
```

- **Whole disk**: **2000 blocks** total (`FSSIZE = 2000`).
- **Log (journal)**: **30 blocks** (blocks 2 through 31; block 31 marks the end of the log area).
- **Inodes**: **13 blocks** (holding $13 \times \frac{1024}{64} = 208$ disk inodes).
- **Bitmap**: **1 block** (block 45, free data block bitmap).
- **Boot and Superblock**: **2 blocks** (boot block 0, superblock 1).
- **Total metadata (`nmeta`)**: **47 blocks** ($1 + 1 + 30 + 13 + 1 = 46$ plus sector 46 bitmap write).
- **Data blocks**: **1953 blocks** (blocks 47 through 1999).

5 Part 4: Write `find`

5.1 Implementation of `user/find.c`

We implemented the recursive directory traversal utility `user/find.c`:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"

void
find(char *path, char *target)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, 0)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }
  if(st.type != T_DIR){
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(strcmp(p, ".") == 0 || strcmp(p, "..") == 0)
      continue;
    if(strcmp(p, target) == 0)
      printf("%s\n", buf);
    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }
    if(st.type == T_DIR)
      find(buf, target);
  }
  close(fd);
```

```
}

int
main(int argc, char *argv[])
{
  if(argc != 3){
    fprintf(2, "usage: find <dir> <name>\n");
    exit(1);
  }
  find(argv[1], argv[2]);
  exit(0);
}
```

5.2 Design Rationale and Safety Measures

1. **Cycle Prevention**: Explicit checks `strcmp(p, ".") == 0 || strcmp(p, "..") == 0` prevent infinite recursive loops through self and parent links.
2. **Free Slot Filtering**: Checks `de.inum == 0` to bypass unallocated directory entries.
3. **File Descriptor Management**: Closes the directory file descriptor (`close(fd)`) immediately before returning. Because xv6 processes are restricted to 16 open files (`NOFILE = 16`), failing to close directory descriptors would cause deep traversals to exhaust the process table and fail.
4. **Proper Null Termination**: Uses `p[DIRSIZ] = 0` to ensure strings are null-terminated, as filenames that occupy all 14 characters are not guaranteed to have trailing null bytes in `struct dirent`.

5.3 Execution Traces and Verification

We verified `find` on a test hierarchy in xv6:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

5.4 Stretch Question Analysis

- **Question**: Which of the three `r2` files are the same file as `cat`?
- **Answer**: `./r2` (in the current directory).
  `./r2` was created with `ln cat r2`, which creates a hard link pointing directly to the inode of `cat` (inode 4). Both `cat` and `./r2` share the exact same inode. In contrast, `./a/r2` and `./a/b/r2` were created using shell output redirection (`echo > ...`), causing xv6 to allocate completely new, distinct inodes.

6 Conclusions

Laboratory Work Nr. 6 provided comprehensive insights into the low-level implementation of the UNIX filesystem architecture through hands-on development in xv6:

1. **Minimalist Elegance of UNIX Directory Architecture**: Treating directories as plain files containing 16-byte fixed records (`ushort inum` and `char name[14]`) demonstrates that complex hierarchical namespaces can be implemented with straightforward linear data structures.

2. **Hard Link Decoupling**: Hard links demonstrate the fundamental UNIX principle separating human-readable path identifiers from physical storage inodes. The unlinking process in `sys_unlink()` safely decrements link counts while deferring storage reclamation until all active descriptors are closed.

3. **Structural Limits of Filesystems**: Filesystem dimensions—such as the 14-character filename limit (`DIRSIZ`) and the 268 KiB file capacity ceiling—are direct consequences of fixed-size metadata definitions (`struct dinode` with 12 direct addresses and one indirect block pointer).

4. **Robust Tree Traversal**: Writing `find` highlighted the critical systems programming requirements of recursion safety (filtering `.` and `..`) and disciplined resource reclamation under strict kernel constraints (`NOFILE = 16`).